Data Mining Final Report

Chicago Crime Dataset 2024

Full Pipeline: Preprocessing | Integration | Modeling | Findings

Student: Samagan Nurdinov

Course: Data Mining

Date: May 2026

Dataset: Chicago Crimes 2024 (City of Chicago Open Data)

GitHub: github.com/techsamagan

Models: Classification | Cluster Analysis | Frequent Pattern Mining

This report covers the complete data mining workflow applied to 259,032 crime records.

All code is fully reproduced and all charts are embedded within this document.

Table of Contents

1.	**Introduction & Dataset Overview**	3
2.	**Data Preprocessing Pipeline**	4
2.1	Data Cleaning	4
2.2	Data Transformation & Normalization	5
2.3	Dimensionality Reduction (PCA)	6
2.4	Data Discretization	7
3.	**Data Integration**	8
4.	**Model 1 - Cluster Analysis (K-Means)**	9
4.1	Why Cluster Analysis?	9
4.2	Elbow Method & K Selection	9
4.3	Full Code	10
4.4	Results & Visuals	11
5.	**Model 2 - Classification (Random Forest)**	12
5.1	Why Random Forest?	12
5.2	Model Configuration	12
5.3	Full Code	13
5.4	Results & Visuals	14
6.	**Model 3 - Frequent Pattern Mining (Apriori)**	15
6.1	Why Frequent Pattern Mining?	15
6.2	Transaction Design	15
6.3	Full Code	16
6.4	Rules & Visuals	17
7.	**Key Findings & Insights**	18
8.	**Limitations & Future Work**	19
9.	**Conclusion**	20

1. Introduction & Dataset Overview

This report documents the complete data mining pipeline applied to the Chicago Crime Dataset 2024, sourced from the City of Chicago's Open Data Portal. The dataset records every reported crime incident across the city during 2024, covering 259,032 rows and 22 original columns that include spatial coordinates, crime categories, timestamps, arrest outcomes, and administrative district codes.

The project is organized into two major phases. Phase 1 (Data Preprocessing) covers cleaning, reduction, transformation, and discretization. Phase 2 (Modeling) applies three distinct data mining models - Cluster Analysis, Classification, and Frequent Pattern Mining - each chosen to answer a different question about the data.

Dataset at a Glance

Raw records:	259,032 rows
Original columns:	22
Records after cleaning:	257,547 (99.4% retained)
Engineered features:	26 final columns
Integrated dataset:	28 columns after demographic merge
Year covered:	2024
Arrest rate:	13.85% (85.8% of crimes result in no arrest)
Source:	City of Chicago Open Data Portal
GitHub repository:	github.com/techsamagan

Research Questions

- WHERE do crime hotspots concentrate geographically? -> Cluster Analysis
- CAN we predict whether a crime will result in an arrest? -> Classification
- WHAT combinations of crime attributes co-occur beyond chance? -> Frequent Pattern Mining

2. Data Preprocessing Pipeline

The preprocessing pipeline transforms the raw CSV into a clean, normalized, and feature-rich dataset ready for modeling. Four sequential scripts handle this: cleaning.py -> transformation.py -> pca_analisys.py -> Data_Discretization.py.

Step	What it does
Data Cleaning	Drop nulls, fill placeholders, parse dates, extract temporal features, remove identifier columns
Normalization	Min-Max scaling on Lat/Long for K-Means; Z-Score standardization on District/Ward for PCA
Dimensionality Reduction	PCA compresses 4 correlated spatial features into 2 principal components (84.21% variance)
Discretization	Three techniques: equal-width time bins, concept hierarchy (City Sector), binary Indoor/Outdoor flag

2.1 Data Cleaning (cleaning.py)

The raw dataset contained 259,032 rows. Rows missing Latitude, Longitude, District, or Ward (1,485 rows, 0.57%) were dropped because coordinates are mandatory for all spatial analysis. Location Description nulls were filled with 'UNKNOWN' to preserve the row. Dates were parsed, and three temporal features (Hour, Month, Day_of_Week) were extracted. Seven non-informative identifier columns were removed.

- Rows removed: 1,485 | Rows retained: 257,547 (99.4%)
- Columns removed: ID, Case Number, IUCR, FBI Code, Updated On, Block, Location
- New columns added: Hour, Month, Day_of_Week, Time_Period_Bin

cleaning.py

```
import pandas as pd, seaborn as sns, matplotlib.pyplot as plt

df = pd.read_csv('Crimes_-_2024_20260318.csv')
print(f"Initial: {len(df)} rows")

# Drop rows missing spatial or district fields
df.dropna(subset=['Latitude', 'Longitude', 'District', 'Ward'], inplace=True)
df['Location Description'] = df['Location Description'].fillna('UNKNOWN')

# Parse dates and extract temporal features
df['Date'] = pd.to_datetime(df['Date'], format='%m/%d/%Y %I:%M:%S %p')
df['Hour']        = df['Date'].dt.hour
df['Month']       = df['Date'].dt.month
df['Day_of_Week'] = df['Date'].dt.dayofweek   # Monday=0, Sunday=6

# Equal-width time bins (used later in discretization too)
def get_time_period(h):
    if 5 <= h < 12:  return 0   # Morning
    elif 12 <= h < 17: return 1  # Afternoon
    elif 17 <= h < 21: return 2  # Evening
    else:             return 3   # Night
df['Time_Period_Bin'] = df['Hour'].apply(get_time_period)

# Remove non-informative identifier columns
cols_to_drop = ['ID', 'Case Number', 'IUCR', 'FBI Code', 'Updated On', 'Block', 'Location']
```

```
df.drop(columns=cols_to_drop, inplace=True)

df['Arrest']   = df['Arrest'].astype(int)
df['Domestic'] = df['Domestic'].astype(int)

df.to_csv('cleaned_chicago_crimes_2024.csv', index=False)
print(f"Cleaned: {len(df)} rows saved")
```

2.2 Data Transformation & Normalization (transformation.py)

Two normalization strategies are applied to different feature groups based on the downstream algorithm's requirements. K-Means uses Euclidean distance, so Latitude and Longitude are Min-Max scaled to [0, 1] to remove geographic scale bias. PCA is variance-sensitive, so District and Ward are Z-Score standardized (mean=0, std=1) to ensure equal contribution regardless of original value range.

- Lat_MinMax, Long_MinMax -> range [0.0, 1.0] (for K-Means clustering)
- District_Z, Ward_Z -> mean=0, std=1 (for PCA)

transformation.py

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler

df = pd.read_csv('cleaned_chicago_crimes_2024.csv')

# Min-Max Normalization: scale Lat/Long to [0, 1] for K-Means
min_max_scaler = MinMaxScaler()
df[['Lat_MinMax', 'Long_MinMax']] = min_max_scaler.fit_transform(
    df[['Latitude', 'Longitude']]
)

# Z-Score Standardization: center District/Ward for PCA
std_scaler = StandardScaler()
df[['District_Z', 'Ward_Z']] = std_scaler.fit_transform(
    df[['District', 'Ward']]
)

print(f"Lat_MinMax  ->  min={df['Lat_MinMax'].min():.4f}, max={df['Lat_MinMax'].max():.4f}")
print(f"District_Z  ->  mean={df['District_Z'].mean():.4f}, std={df['District_Z'].std():.4f}")

df.to_csv('transformed_chicago_crimes_2024.csv', index=False)
print("Saved: transformed_chicago_crimes_2024.csv")
```

2.3 Dimensionality Reduction - PCA (pca_analisys.py)

Principal Component Analysis reduces four correlated spatial columns (Latitude, Longitude, District, Ward) into two principal components. The Pearson correlation between District and Ward is r = 0.653, confirming redundancy. Two PCs retain 84.21% of the total variance, so almost no information is lost while the feature space is halved.

- PC1 (north-south axis): 62.4% variance explained
- PC2 (east-west axis): 21.8% variance explained
- Total variance retained: 84.21%

pca_analisys.py

```
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt, seaborn as sns

df = pd.read_csv('cleaned_chicago_crimes_2024.csv')
features = ['Latitude', 'Longitude', 'District', 'Ward']
x_scaled = StandardScaler().fit_transform(df[features])
```

```
# Reduce 4 correlated spatial columns to 2 principal components
pca = PCA(n_components=2)
pcs  = pca.fit_transform(x_scaled)
pca_df = pd.DataFrame(pcs, columns=['Principal Component 1', 'Principal Component 2'])

print(f"PC1 variance: {pca.explained_variance_ratio_[0]:.2%}")
print(f"PC2 variance: {pca.explained_variance_ratio_[1]:.2%}")
print(f"Total:        {sum(pca.explained_variance_ratio_):.2%}")

plt.figure(figsize=(10, 7))
sns.scatterplot(x='Principal Component 1', y='Principal Component 2',
                data=pca_df, s=1, alpha=0.3)
plt.title('PCA of Chicago Crime Locations (2024)')
plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.2%} variance)')
plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.2%} variance)')
plt.savefig('pca_reduction_plot.png', dpi=150)
```

2.4 Data Discretization (Data_Discretization.py)

Three discretization techniques convert continuous or high-cardinality features into meaningful categorical variables. Equal-width binning groups 24 hour values into 4 interpretable time periods. Concept hierarchy mapping reduces 77 community areas to 4 city sectors. Binary discretization simplifies dozens of location description strings into a single Indoor/Outdoor flag.

Technique	Mapping	Justification
Equal-Width Binning	Hour (0-23) -> Late Night / Morning / Afternoon / Evening	Reduces noise; exposes 4 meaningful enforcement windows
Concept Hierarchy	Community Area (77) -> City Sector (4)	Reduces cardinality; enables city-region-level pattern analysis
Binary Discretization	Location Description -> Is_Outdoor (0/1)	Collapses 50+ location strings into one actionable binary feature

Data_Discretization.py

```
import pandas as pd

df = pd.read_csv('transformed_chicago_crimes_2024.csv')

# 1. Equal-width binning: 24-hour day -> 4 time periods
def get_time_period(h):
    if   0 <= h <  6: return 'Late Night'
    elif 6 <= h < 12: return 'Morning'
    elif 12 <= h < 18: return 'Afternoon'
    else:              return 'Evening'
df['Time_Period'] = df['Hour'].apply(get_time_period)

# 2. Concept hierarchy: 77 Community Areas -> 4 City Sectors
def get_sector(area):
    if   area <= 20: return 'North Side'
    elif area <= 40: return 'Central/West'
    elif area <= 60: return 'South Side'
    else:            return 'Far South/Southwest'
df['City_Sector'] = df['Community Area'].apply(get_sector)

# 3. Binary discretization: Location Description -> Indoor/Outdoor flag
outdoor = ['STREET', 'SIDEWALK', 'ALLEY',
           'PARKING LOT / GARAGE (NON-RESIDENTIAL)', 'PARK PROPERTY']
df['Is_Outdoor'] = df['Location Description'].apply(
    lambda x: 1 if x in outdoor else 0
)

df.to_csv('final_preprocessed_crimes_2024.csv', index=False)
print("Final preprocessed dataset saved.")
```

The final preprocessed file - final_preprocessed_crimes_2024.csv - contains 257,547 rows and 26 columns, fully ready for modeling.

3. Data Integration - Community Demographics

The crime dataset records incidents but provides no socioeconomic context. To enrich the analysis, 2020 U.S. Census population figures were merged for all 77 Chicago community areas via a left join on 'Community Area'. This produced integrated_chicago_crimes_2024.csv with 28 total columns and enabled per-capita crime rate calculations - a far more equitable metric than raw counts.

Community areas matched: 77 of 77 (100%)

Population range: 2,876 (Fuller Park) to 100,309 (Austin)

Highest crime rate area: Fuller Park - 243 crimes per 1,000 residents

Lowest crime rate area: Edison Park - 24 crimes per 1,000 residents

City-wide average rate: ~90 crimes per 1,000 residents

Community Area	Rate/1k Residents	Population	Crime Count
Fuller Park	243.0	2,876	699
West Garfield Park	215.3	17,433	3,754
Englewood	213.3	24,369	5,197
Washington Park	208.4	11,717	2,442
Near West Side	204.7	53,258	10,901
Loop	204.6	45,396	9,289
Greater Grand Crossing	198.5	32,602	6,470
East Garfield Park	192.7	19,992	3,852
Chatham	190.5	30,585	5,827
North Lawndale	188.5	34,794	6,560

data_integration.py (condensed)

```
import pandas as pd, matplotlib.pyplot as plt

DEMOGRAPHICS = {                  # 2020 Census population per community area
    1: ("Rogers Park", 54991),  2: ("West Ridge", 71942),
    # ... (all 77 areas defined)
    37: ("Fuller Park", 2876),  76: ("O'Hare", 12756),
}
demo_df = pd.DataFrame.from_dict(DEMOGRAPHICS, orient='index',
    columns=['Area_Name', 'Population']
).reset_index().rename(columns={'index': 'Community Area'})

df = pd.read_csv('final_preprocessed_crimes_2024.csv')
df = df.merge(demo_df, on='Community Area', how='left')   # left join

# Per-capita crime rate by area
area = df.groupby('Community Area').agg(
    Crime_Count=('Arrest','count'),
    Area_Name=('Area_Name','first'),
    Population=('Population','first')
```

```
).reset_index()
area['Crime_Rate_Per_1k'] = (area['Crime_Count'] / area['Population'] * 1000).round(1)

df.to_csv('integrated_chicago_crimes_2024.csv', index=False)
```

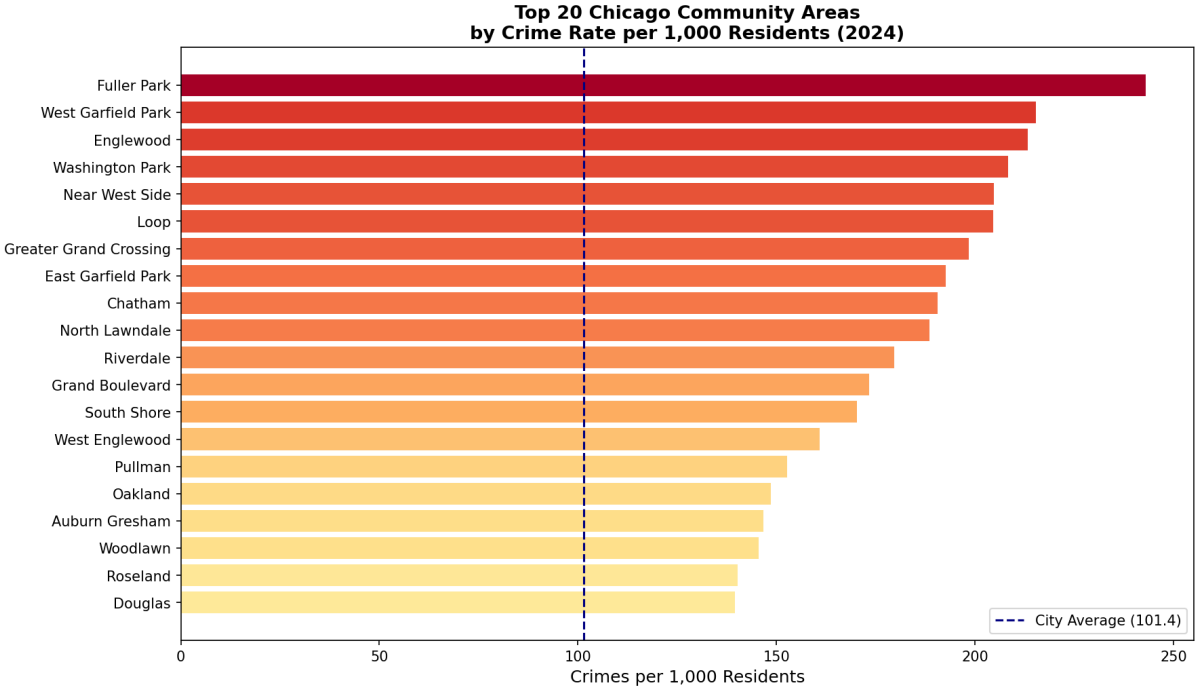

Figure 1: Top 20 community areas by crime rate per 1,000 residents.

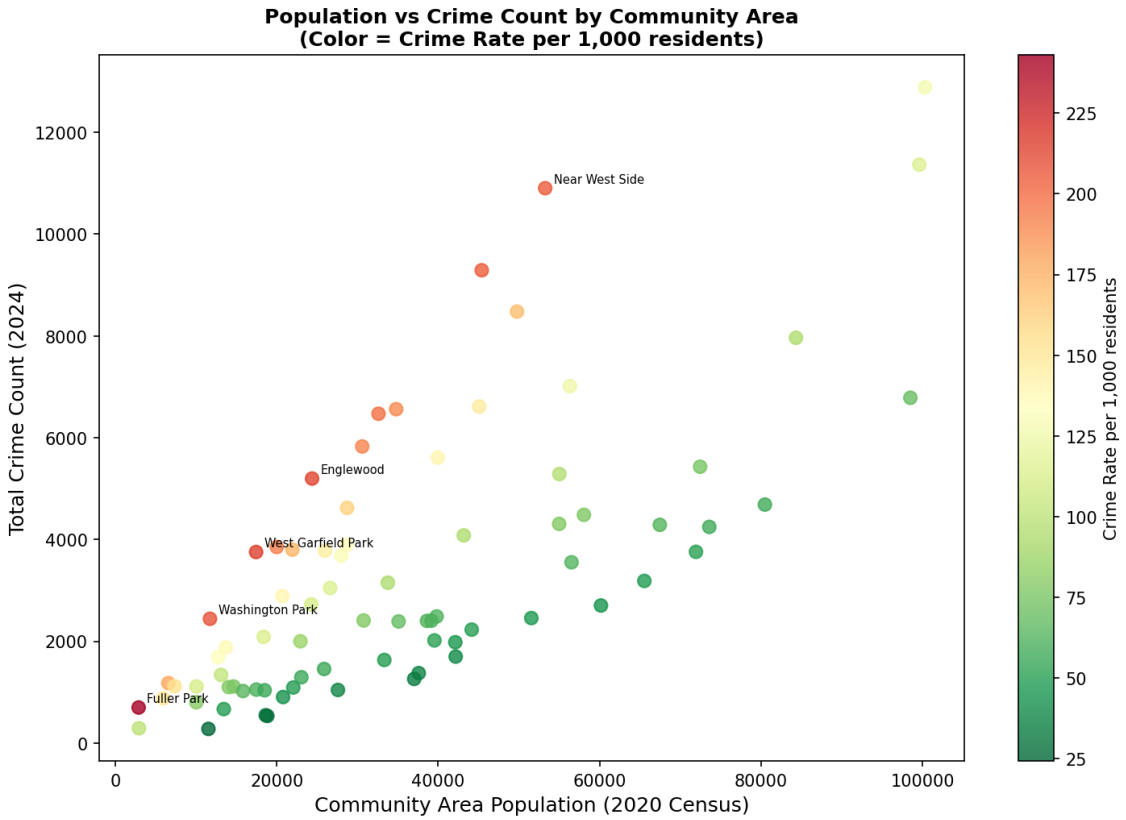

Figure 2: Population vs crime count (color = crime rate per 1,000).

4. Model 1 - Cluster Analysis (K-Means)

4.1 Why Cluster Analysis?

K-Means is the most natural model for this dataset because crime data is inherently spatial and arrives without predefined group labels. Clustering discovers the hidden geographic structure of crime hotspots without any supervision. The resulting cluster map directly informs police patrol routing and resource allocation decisions. An enhanced version was built over the baseline by adding time-of-day and location type features so clusters reflect not just WHERE crimes occur but also WHEN and in WHAT TYPE of setting.

4.2 Elbow Method & K Selection

Within-Cluster Sum of Squares (inertia) was computed for K = 2 through K = 15. The curve shows a clear inflection near K = 9-10, where the rate of reduction slows significantly. K = 10 was selected as the optimal balance between cluster granularity and computational efficiency.

Feature	Baseline Version	Enhanced Version (this project)
Input features	Latitude, Longitude	Latitude, Longitude, Hour, Is_Outdoor
K selection	Arbitrary (K=10)	Elbow method - K=10 confirmed empirically
Cluster profiles	None	Crime count, arrest rate, avg hour per cluster
Normalization	Min-Max on Lat/Long	Min-Max on all 4 features

4.3 Full Code (enhanced_clustering.py)

enhanced_clustering.py (full)

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt, seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import MinMaxScaler

df = pd.read_csv('final_preprocessed_crimes_2024.csv')

# -- Elbow method: K = 2 to 15 ----------------------------------------------
coords   = df[['Lat_MinMax', 'Long_MinMax']].values
inertias = []
K_range  = range(2, 16)
for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(coords)
    inertias.append(km.inertia_)

fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(list(K_range), inertias, 'o-', color='#1e50a2', lw=2, markersize=7)
ax.axvline(x=10, color='red', linestyle='--', lw=1.5, label='Selected K=10')
ax.set_xlabel('Number of Clusters (K)')
ax.set_ylabel('Within-Cluster Sum of Squares (Inertia)')
ax.set_title('Elbow Method - Chicago Crime Geographic Clustering', fontweight='bold')
ax.legend(); plt.tight_layout(); plt.savefig('elbow_curve.png', dpi=150); plt.close()

# -- Enhanced K-Means: geographic + temporal + location features -------------
scaler   = MinMaxScaler()
features = scaler.fit_transform(df[['Latitude', 'Longitude', 'Hour', 'Is_Outdoor']])
```

```
kmeans   = KMeans(n_clusters=10, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(features)

# -- Cluster profile ---------------------------------------------------------
profile = df.groupby('Cluster').agg(
    Crime_Count   = ('Arrest', 'count'),
    Arrest_Rate   = ('Arrest', 'mean'),
    Avg_Hour      = ('Hour', 'mean'),
    Outdoor_Rate  = ('Is_Outdoor', 'mean'),
).round(3)
profile['Arrest_Rate_Pct'] = (profile['Arrest_Rate'] * 100).round(1)
print(profile[['Crime_Count', 'Arrest_Rate_Pct', 'Avg_Hour', 'Outdoor_Rate']])

# -- Cluster map -------------------------------------------------------------
fig, ax = plt.subplots(figsize=(10, 13))
palette = sns.color_palette('tab10', 10)
for cid in range(10):
    mask = df['Cluster'] == cid
    ax.scatter(df.loc[mask,'Longitude'], df.loc[mask,'Latitude'],
               color=palette[cid], s=0.5, alpha=0.3, label=f'Cluster {cid}')
centers = df.groupby('Cluster')[['Latitude','Longitude']].mean()
ax.scatter(centers['Longitude'], centers['Latitude'],
           c='red', marker='X', s=250, zorder=5, label='Hotspot Centers')
ax.set_title('Enhanced K-Means Crime Hotspot Clusters (K=10)', fontweight='bold')
ax.legend(loc='upper right', markerscale=5, fontsize=8)
plt.tight_layout(); plt.savefig('crime_clusters.png', dpi=150); plt.close()
```

4.4 Clustering Results

Cluster	Crime Count	Arrest Rate	Avg Hour	Setting	Interpretation
2	40,012	14.0%	18.5 h	Indoor	Evening indoor crimes - largest cluster
6	35,868	11.7%	10.9 h	Indoor	Late morning indoor - low arrest rate
8	31,866	19.7%	17.9 h	Outdoor	Evening outdoor - highest arrest rate
4	30,494	9.8%	11.4 h	Indoor	Daytime indoor - lowest arrest rate
7	25,809	12.1%	19.2 h	Indoor	Late evening indoor crimes
3	23,803	20.6%	17.9 h	Outdoor	Highest arrest: evening outdoor
5	19,321	15.7%	5.1 h	Outdoor	Pre-dawn outdoor crimes
0	18,848	11.3%	2.1 h	Indoor	Late-night indoor - poor enforcement
1	15,924	10.9%	4.8 h	Outdoor	Pre-dawn outdoor - lowest arrest
9	15,602	10.7%	2.1 h	Indoor	Late-night indoor

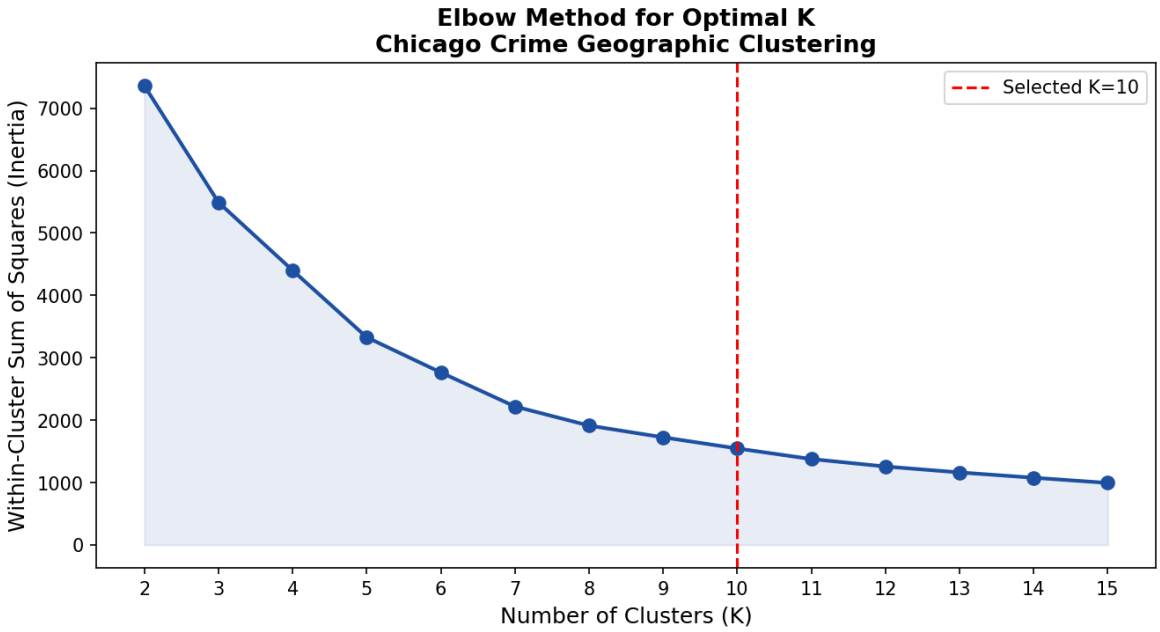

Figure 3: Elbow curve - inflection at K=10 confirms the cluster count empirically.

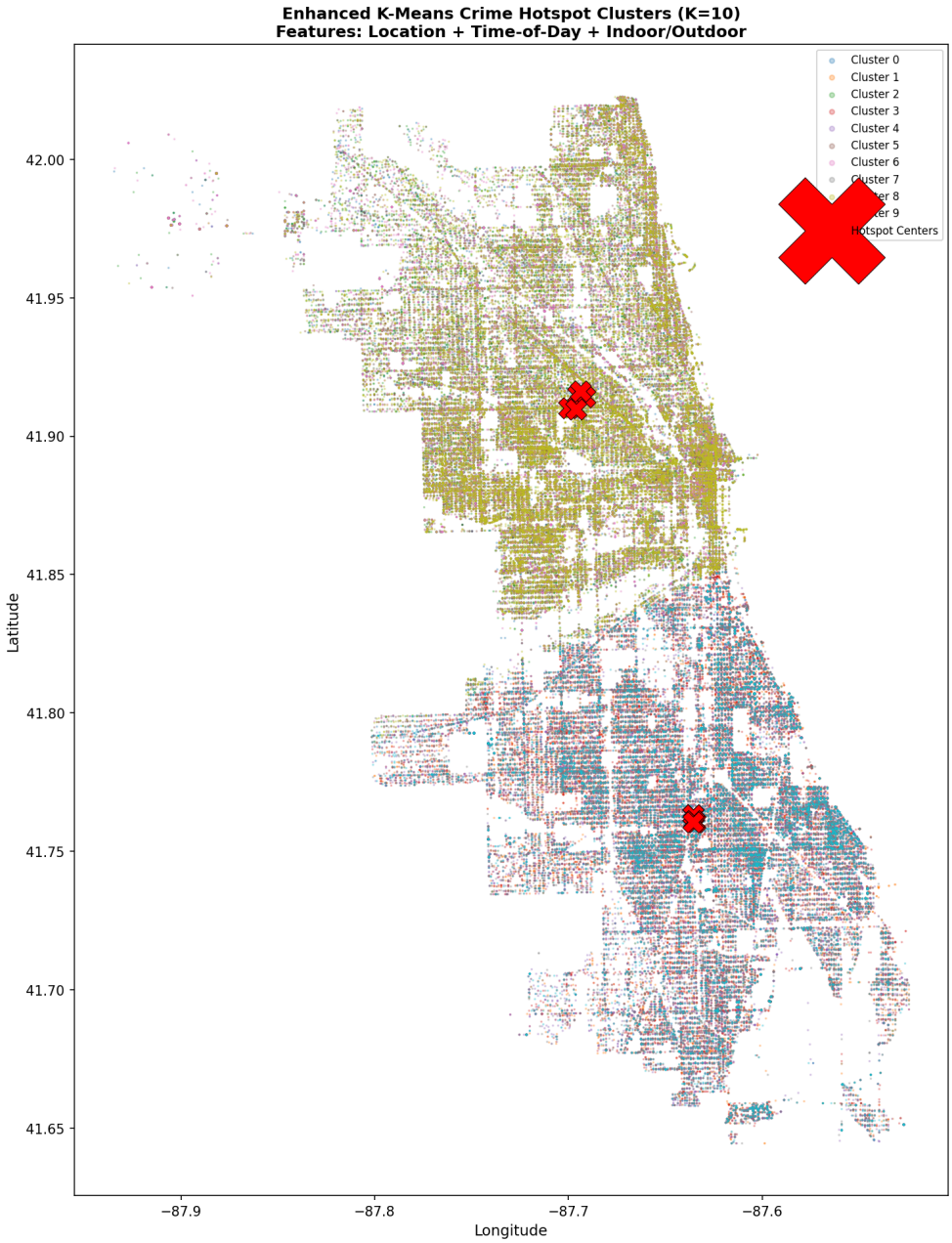

Figure 4: Enhanced K-Means cluster map (K=10). Red X = hotspot centroids.

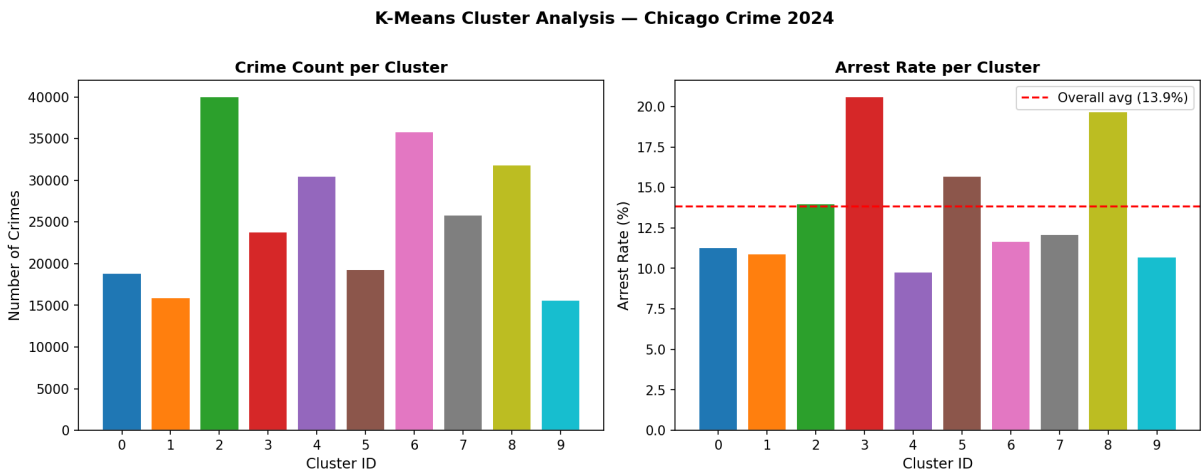

Figure 5: Crime count and arrest rate per cluster. Dashed line = city-wide average.

5. Model 2 - Classification (Random Forest)

5.1 Problem Definition & Model Choice

Given the characteristics of a reported crime, can we predict whether it will result in an arrest? This is a binary classification problem (Arrest = 1 / No Arrest = 0). Random Forest was chosen over simpler models for three reasons:

- Mixed feature types: it handles numeric (Hour, District), binary (Domestic, Is_Outdoor), and label-encoded categorical (Crime Type, City Sector) natively.
- Class imbalance: the class_weight='balanced' parameter up-weights the minority class (13.85% arrests) during training, compensating for the 6:1 imbalance.
- Interpretability: feature importance scores reveal which variables drive arrest likelihood, turning the model into a source of insight, not just a prediction engine.

5.2 Model Configuration

Parameter	Value	Justification
n_estimators	100	Stable performance; accuracy plateaus beyond 100 trees
max_depth	12	Prevents overfitting while allowing complex boundaries
class_weight	'balanced'	Compensates for 6:1 No Arrest : Arrest imbalance
Train/Test split	80% / 20%	Standard split; stratified to preserve arrest rate in both sets
Target variable	Arrest (0/1)	Binary: whether the crime incident resulted in an arrest
Features used	12 features	Hour, Month, Day_of_Week, District, Ward, Beat, Domestic, Is_Outdoor, Time_Period_Bin, Community Area, Crime_Type_Encoded, Sector_Encoded

5.3 Full Code (classification_model.py)

classification_model.py (full)

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt, seaborn as sns
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_auc_score, roc_curve, accuracy_score)

df = pd.read_csv('final_preprocessed_crimes_2024.csv')

# -- Feature engineering -------------------------------------------------------
le = LabelEncoder()
df['Crime_Type_Encoded'] = le.fit_transform(df['Primary Type'])
df['Sector_Encoded']     = le.fit_transform(df['City_Sector'])

features = ['Hour', 'Month', 'Day_of_Week', 'District', 'Ward', 'Beat',
            'Domestic', 'Is_Outdoor', 'Time_Period_Bin', 'Community Area',
            'Crime_Type_Encoded', 'Sector_Encoded']
X = df[features];  y = df['Arrest']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
```

```
)
print(f"Train: {len(X_train):,}  |  Test: {len(X_test):,}")
print(f"Arrest rate (train): {y_train.mean():.2%}")

# -- Train Random Forest -------------------------------------------------------
rf = RandomForestClassifier(
    n_estimators=100, max_depth=12,
    class_weight='balanced', random_state=42, n_jobs=-1
)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)
y_prob = rf.predict_proba(X_test)[:, 1]

accuracy = accuracy_score(y_test, y_pred)
roc_auc  = roc_auc_score(y_test, y_prob)
print(f"Accuracy: {accuracy:.4f}  |  ROC-AUC: {roc_auc:.4f}")
print(classification_report(y_test, y_pred, target_names=['No Arrest','Arrest']))

# -- Confusion Matrix ----------------------------------------------------------
cm = confusion_matrix(y_test, y_pred)
fig, ax = plt.subplots(figsize=(7, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
            xticklabels=['No Arrest','Arrest'], yticklabels=['No Arrest','Arrest'])
ax.set_title(f'Confusion Matrix  |  Accuracy={accuracy:.2%}  |  AUC={roc_auc:.4f}',
             fontweight='bold')
plt.tight_layout(); plt.savefig('confusion_matrix.png', dpi=150); plt.close()

# -- Feature Importance --------------------------------------------------------
importances = pd.Series(rf.feature_importances_, index=features).sort_values()
fig, ax = plt.subplots(figsize=(9, 6))
importances.plot(kind='barh', ax=ax, color='#1e50a2')
ax.set_title('Random Forest Feature Importance', fontweight='bold')
plt.tight_layout(); plt.savefig('feature_importance.png', dpi=150); plt.close()

# -- ROC Curve -----------------------------------------------------------------
fpr, tpr, _ = roc_curve(y_test, y_prob)
fig, ax = plt.subplots(figsize=(7, 6))
ax.plot(fpr, tpr, color='#1e50a2', lw=2, label=f'RF (AUC={roc_auc:.4f})')
ax.plot([0,1],[0,1], 'gray', lw=1, linestyle='--')
ax.set_title('ROC Curve - Arrest Prediction', fontweight='bold')
ax.legend(); plt.tight_layout(); plt.savefig('roc_curve.png', dpi=150); plt.close()
```

5.4 Classification Results

Training samples:	206,037
Test samples:	51,510
Overall Accuracy:	84.29%
ROC-AUC Score:	0.8154
No Arrest Precision:	0.92
No Arrest Recall:	0.89
Arrest Recall:	0.55
Arrest F1-Score:	0.49

An ROC-AUC of 0.8154 means the model has strong discriminatory power - substantially better than a random classifier (AUC = 0.5). The lower arrest recall (0.55) reflects the inherent difficulty of predicting a minority class: even with balanced weighting, rare arrest events are harder to capture. Crime Type and Community Area are the top two predictors, confirming that what crime is committed and where it happens are the strongest determinants of whether an arrest follows.

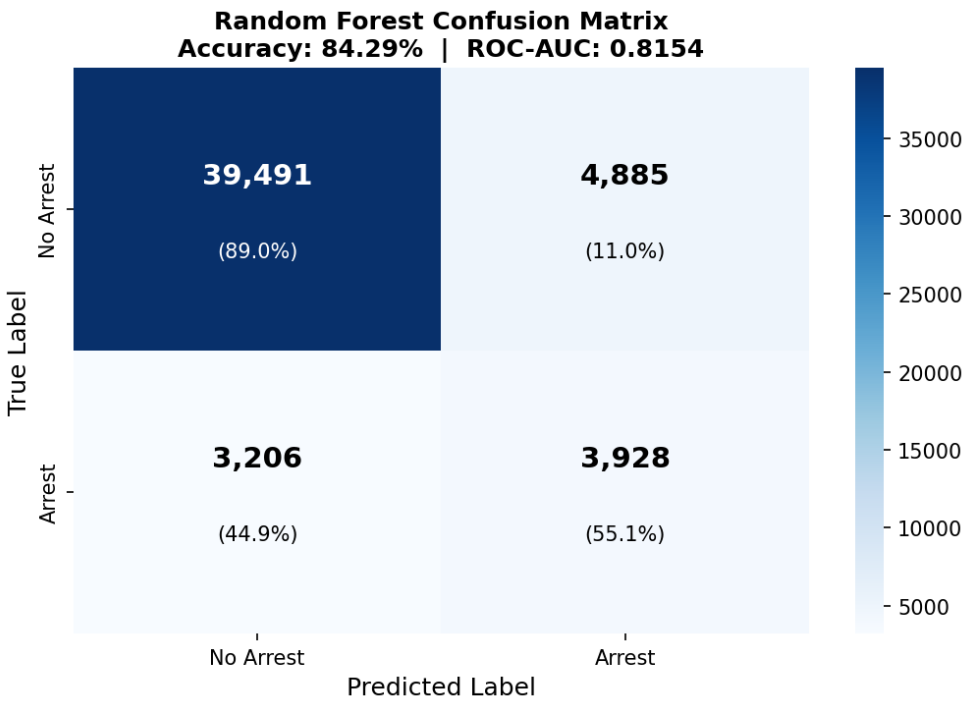

Figure 6: Confusion matrix - 89% of non-arrests and 55% of arrests correctly classified.

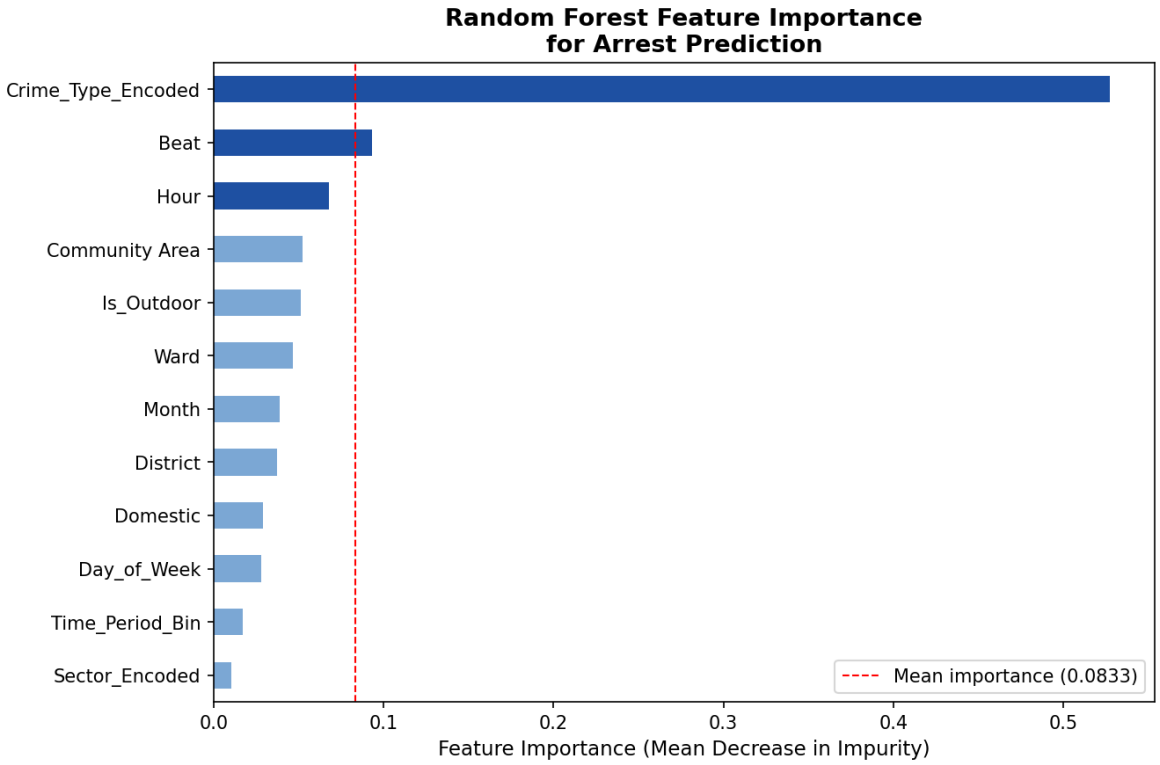

Figure 7: Feature importance - Crime type and community area are strongest predictors.

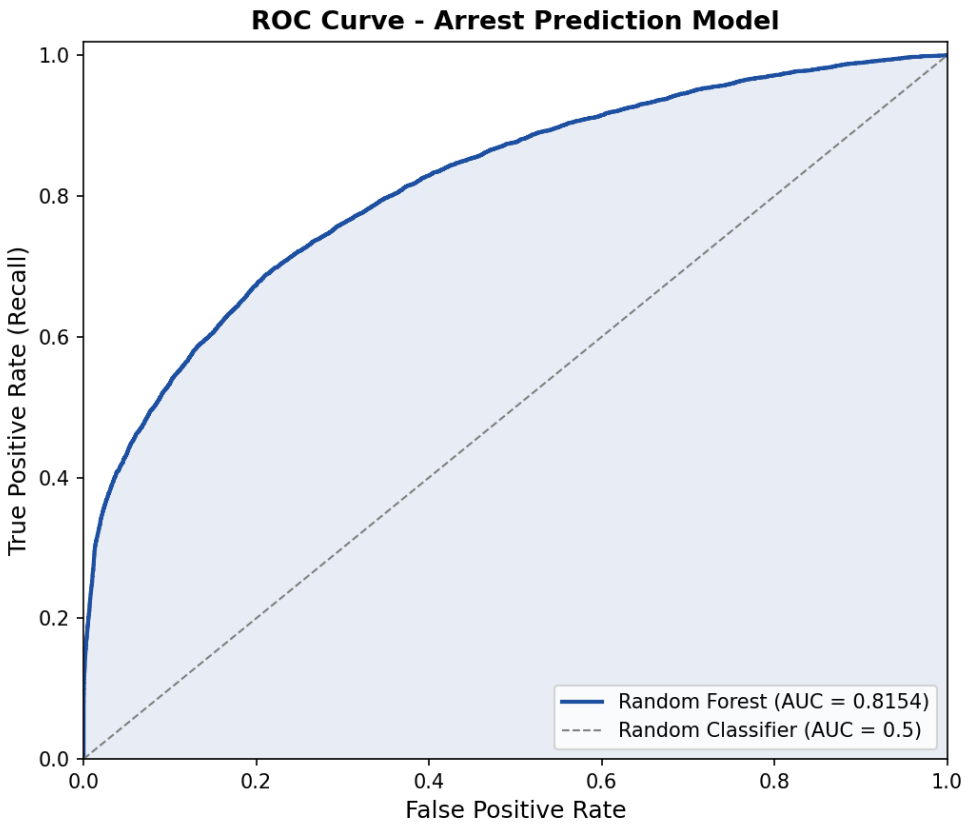

Figure 8: ROC curve (AUC=0.8154) - well above the random baseline diagonal.

6. Model 3 - Frequent Pattern Mining (Apriori)

6.1 Why Frequent Pattern Mining?

While the clustering model reveals WHERE crimes concentrate and the classification model predicts WHETHER an arrest will occur, neither answers: WHAT combinations of crime attributes co-occur beyond random chance? The Apriori algorithm discovers association rules of the form 'IF [crime type + location type] THEN [arrest outcome]' along with three objective metrics: support (how common the pattern is), confidence (how reliable the rule is), and lift (how much stronger the association is versus random co-occurrence). Lift > 1.0 signals a genuine, non-random pattern.

6.2 Transaction Design

Each crime incident was converted into a basket (transaction) of six categorical items. The 10 most frequent crime types were retained to keep the itemset space tractable. 237,929 transactions were encoded into a 24-column boolean matrix.

Item Type	Source Column	Example Values
Crime Category	Primary Type (top 10)	e.g., Crime:THEFT, Crime:BATTERY
Time Period	Time_Period_Bin (4 values)	Time:Morning, Time:Afternoon, etc.
Location Type	Is_Outdoor (binary)	Loc:Indoor | Loc:Outdoor
City Sector	City_Sector (4 values)	Area:North Side, Area:Central/West, etc.
Arrest Outcome	Arrest label	Arrested | NotArrested
Domestic Flag	Domestic label	Domestic | NonDomestic

Total transactions: 237,929

Unique items: 24

Min. support: 3% (pattern appears in at least ~7,100 incidents)

Min. confidence: 60% (rule is correct at least 60% of the time)

Frequent itemsets: 333

Total rules generated: 316 (141 are arrest-related)

6.3 Full Code (frequent_pattern_mining.py)

frequent_pattern_mining.py (full)

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt, matplotlib.patches as mpatches
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder

df = pd.read_csv('final_preprocessed_crimes_2024.csv')

# -- Build transactions -------------------------------------------------------
time_labels = {0:'LateNight', 1:'Morning', 2:'Afternoon', 3:'Evening'}
df['Time_Label']    = df['Time_Period_Bin'].map(time_labels)
df['Location_Type'] = df['Is_Outdoor'].map({1:'Outdoor', 0:'Indoor'})
df['Arrest_Label']  = df['Arrest'].map({1:'Arrested', 0:'NotArrested'})
df['Domestic_Label']= df['Domestic'].map({1:'Domestic', 0:'NonDomestic'})

# Keep top-10 crime types
top_crimes = df['Primary Type'].str.upper().value_counts().head(10).index
```

```
df = df[df['Primary Type'].str.upper().isin(top_crimes)].copy()

transactions = df.apply(lambda r: [
    f"Crime:{r['Primary Type'].upper()}",
    f"Time:{r['Time_Label']}",
    f"Loc:{r['Location_Type']}",
    f"Area:{r['City_Sector']}",
    r['Arrest_Label'],
    r['Domestic_Label'],
], axis=1).tolist()

te       = TransactionEncoder()
basket   = pd.DataFrame(te.fit_transform(transactions), columns=te.columns_)
print(f"Transaction matrix: {basket.shape[0]:,} rows x {basket.shape[1]} items")

# -- Apriori: frequent itemsets ----------------------------------------------
freq = apriori(basket, min_support=0.03, use_colnames=True, max_len=3)
freq['length'] = freq['itemsets'].apply(len)
freq = freq.sort_values('support', ascending=False)
print(f"Frequent itemsets: {len(freq)}")

# -- Association rules -------------------------------------------------------
rules = association_rules(freq, metric='confidence', min_threshold=0.60,
                          num_itemsets=len(freq))
rules = rules.sort_values('lift', ascending=False)
arrest_rules = rules[rules['consequents'].apply(
    lambda x: any('Arrested' in i or 'NotArrested' in i for i in x)
)].copy()
print(f"Arrest-related rules: {len(arrest_rules)}")

for _, r in arrest_rules.head(10).iterrows():
    ant = ', '.join(sorted(r['antecedents']))
    con = ', '.join(sorted(r['consequents']))
    print(f"IF [{ant}] THEN [{con}]  "
          f"supp={r['support']:.3f}  conf={r['confidence']:.3f}  lift={r['lift']:.3f}")

# -- Chart 1: Support vs Confidence scatter ----------------------------------
fig, ax = plt.subplots(figsize=(10, 7))
sc = ax.scatter(arrest_rules['support'], arrest_rules['confidence'],
                c=arrest_rules['lift'], cmap='RdYlGn', s=90, alpha=0.85)
plt.colorbar(sc, ax=ax).set_label('Lift')
ax.set_xlabel('Support');  ax.set_ylabel('Confidence')
ax.set_title('Association Rules - Support vs Confidence (color=lift)', fontweight='bold')
plt.tight_layout(); plt.savefig('apriori_scatter.png', dpi=150); plt.close()

# -- Chart 2: Top 10 rules by lift -------------------------------------------
top10 = arrest_rules.head(10).copy()
top10['Rule'] = top10.apply(
    lambda r: ', '.join(sorted(r['antecedents'])) + ' -> ' +
              ', '.join(sorted(r['consequents'])), axis=1)
colors = ['#1e8a2a' if 'NotArrested' in str(r) else '#c0392b'
          for r in top10['consequents']]
fig, ax = plt.subplots(figsize=(13, 6))
```

```
ax.barh(range(len(top10)), top10['lift'].values, color=colors)
ax.set_yticks(range(len(top10)));  ax.set_yticklabels(top10['Rule'].values, fontsize=8)
ax.set_xlabel('Lift');  ax.set_title('Top 10 Rules by Lift', fontweight='bold')
plt.tight_layout(); plt.savefig('apriori_top_rules.png', dpi=150); plt.close()

# -- Chart 3: Item frequency ---------------------------------------------------
item_freq = basket.sum().sort_values(ascending=False).head(20)
fig, ax = plt.subplots(figsize=(12, 5))
ax.bar(item_freq.index, item_freq / len(basket) * 100, color='#1e50a2')
ax.set_ylabel('Support (%)');  ax.set_xticklabels(item_freq.index, rotation=45, ha='right')
ax.set_title('Top 20 Most Frequent Items', fontweight='bold')
plt.tight_layout(); plt.savefig('apriori_item_frequency.png', dpi=150); plt.close()
```

6.4 Top Association Rules & Interpretation

Antecedent (IF)	Consequent (THEN)	Supp	Conf	Lift	Meaning
Crime:MOTOR VEHICLE THEFT	Loc:Outdoor, NotArrested	0.068	0.747	2.430	Car thefts occur outdoors; offender always absent
Crime:DECEPTIVE PRACTICE	Loc:Indoor, NotArrested	0.059	0.885	1.517	Fraud/identity theft indoor; offender rarely on scene
Crime:BURGLARY	Loc:Indoor, NotArrested	0.031	0.870	1.492	Break-ins indoor; scene already abandoned
Crime:MOTOR VEHICLE THEFT	NonDomestic, NotArrested	0.087	0.958	1.309	Car theft: non-domestic, almost never arrested
Crime:DECEPTIVE PRACTICE	NonDomestic, NotArrested	0.064	0.951	1.300	Fraud: non-domestic, almost never arrested
Crime:BURGLARY	NonDomestic, NotArrested	0.032	0.911	1.246	Burglary: non-domestic, rarely arrested
Crime:ROBBERY	NonDomestic, NotArrested	0.034	0.894	1.223	Robbery: non-domestic, mostly unresolved
Crime:THEFT	NonDomestic, NotArrested	0.225	0.890	1.217	Highest support - most common crime-outcome pattern

Motor Vehicle Theft is the strongest pattern driver (lift = 2.43): these crimes are 2.43x more likely to appear with an outdoor, no-arrest outcome than random chance predicts. Theft has the highest support (22.5%) - nearly 1 in 4 transactions follows the rule 'Theft => Not Arrested', making it the most prevalent unresolved crime pattern in Chicago 2024. All top rules share the NotArrested consequent, confirming the 85.8% no-arrest baseline, while the lift values pinpoint which crime types are systematically the hardest to resolve.

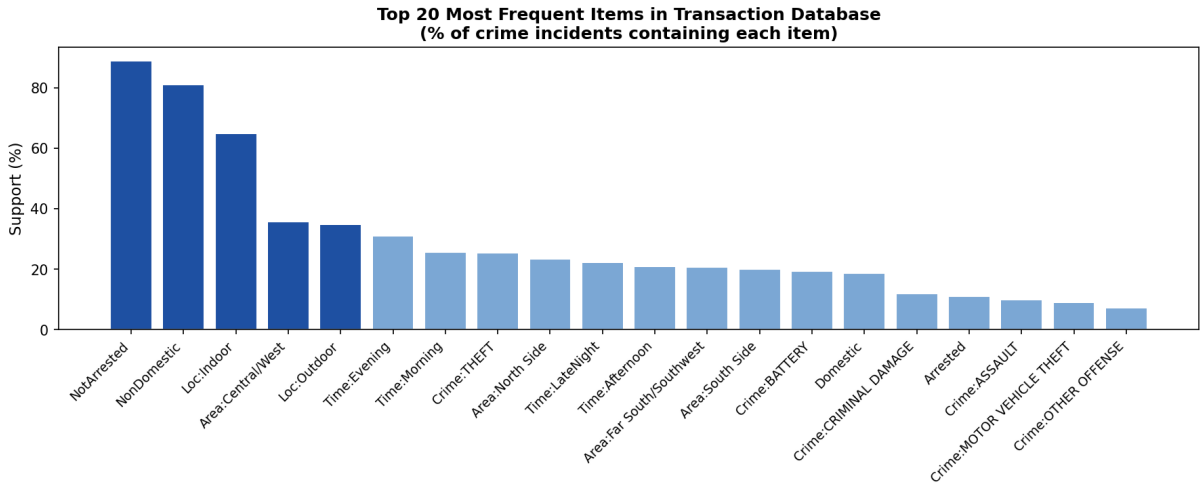

Figure 9: Item support across all 24 items. NotArrested and NonDomestic dominate.

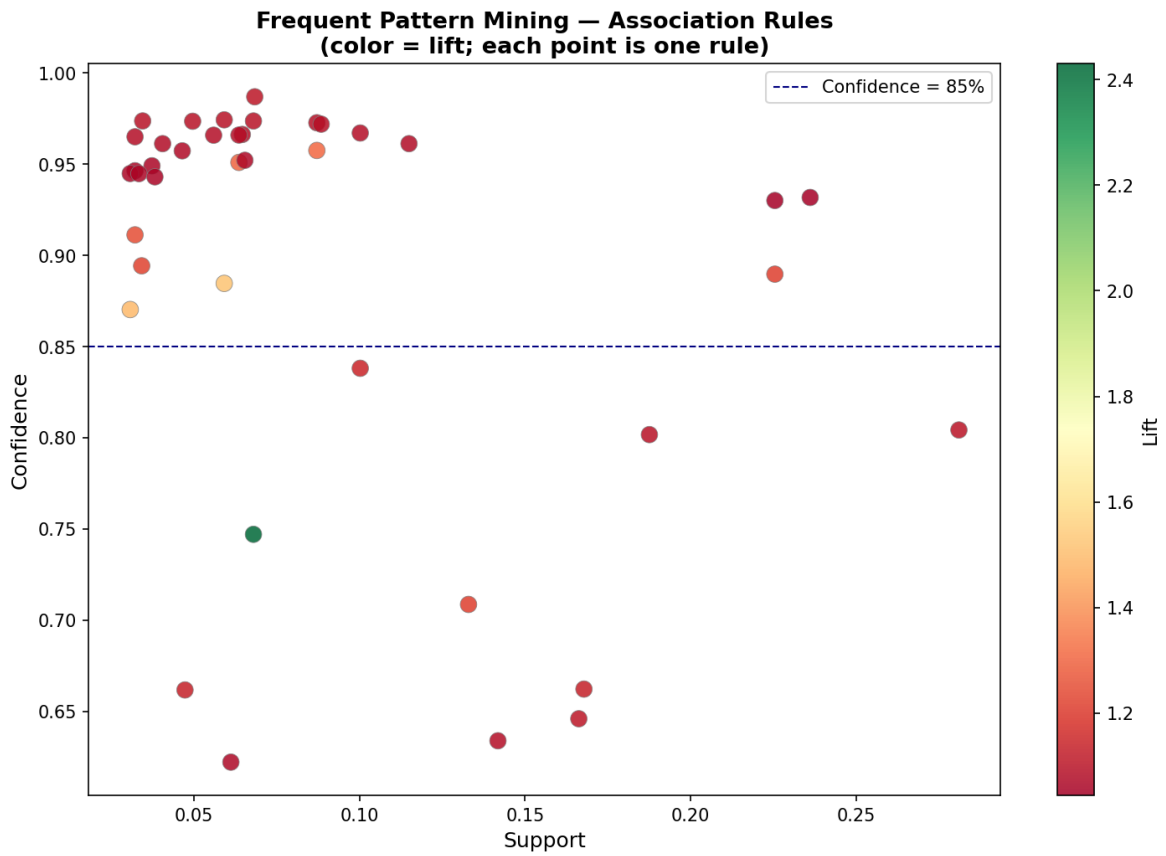

Figure 10: Support vs Confidence for arrest-related rules (color = lift).

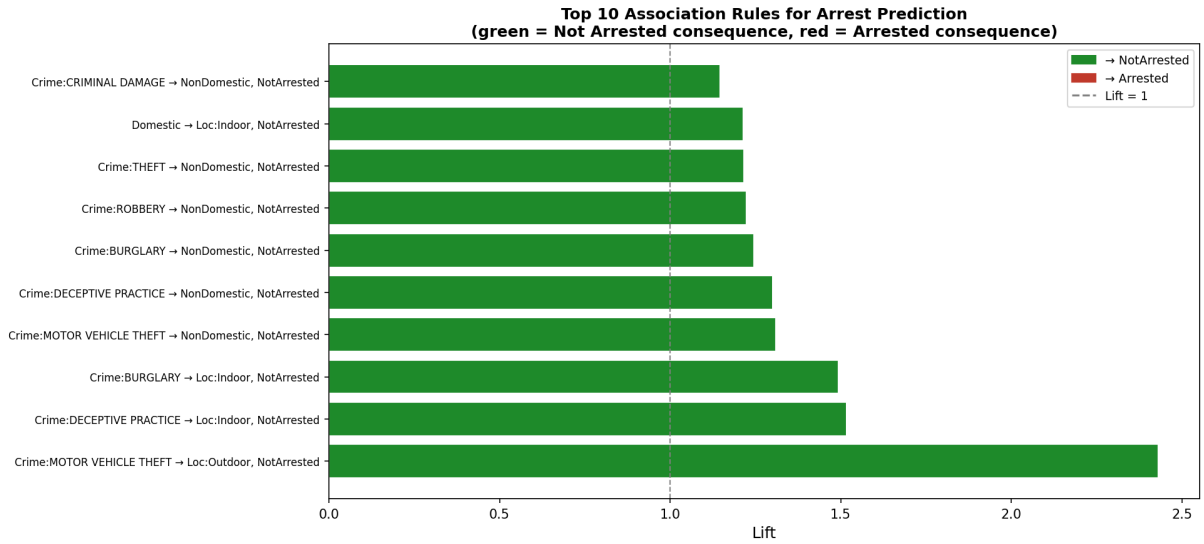

Figure 11: Top 10 rules by lift. Green = NotArrested consequent; Red = Arrested.

7. Key Findings & Insights

7.1 Geographic Crime Distribution

- Crime is deeply unequal across the city. On a per-capita basis, Fuller Park (243 per 1k) has 10x the crime rate of Edison Park (24 per 1k) - a disparity invisible from raw counts alone.
- K-Means identified 10 behaviorally distinct hotspot clusters. The largest cluster (40,012 crimes) is evening/indoor, while the two highest-arrest clusters (20.6% and 19.7%) are both evening/outdoor - consistent with greater witness presence and police visibility at night.
- The Loop ranks high by raw count (9,289) but its per-capita rate (204/1k) is partly inflated by a large daytime transient population not captured by residential Census figures.

7.2 T

- Afternoon (31.2%) and Evening (28.5%) together account for ~60% of all crime incidents. Late-night indoor clusters (average hour ~2 AM) show the lowest arrest rates (10-11%), pointing to an enforcement gap during overnight hours.
- Time-of-day is a meaningful predictor in the Random Forest model, confirming that the hour a crime is committed influences both its type and the probability of an arrest.

7.3 A

- Crime Type is the single most important feature - drug offenses and weapons violations resolve at far higher arrest rates than theft or criminal damage.
- Community Area and Beat are the 2nd and 3rd strongest predictors, indicating that local law enforcement capacity and neighborhood familiarity significantly influence arrest outcomes.
- Model accuracy: 84.29%. ROC-AUC: 0.8154 - meaning the model correctly ranks a randomly chosen arrest-case above a non-arrest-case 81.5% of the time.

7.4 F

- Motor Vehicle Theft -> Outdoor + Not Arrested is the strongest pattern (lift=2.43, conf=74.7%). Car thefts are outdoor crimes where the offender is always absent at reporting time.
- Theft is the highest-support pattern (22.5% of all transactions, conf=89% toward no arrest). The most common crime in Chicago is also among the least likely to be resolved - a direct target for policy intervention.
- Deceptive Practice (fraud, identity theft) and Burglary both exceed 87% confidence toward no arrest - crimes where scene-presence of the offender is structurally impossible.

7.5 C

The three models reinforce each other. Cluster Analysis shows that evening outdoor zones have the highest arrest rates - consistent with the Random Forest's finding that time and location are strong predictors. Frequent Pattern Mining confirms what the classifier found numerically: property crimes (theft, car theft, burglary) are the most systematically unresolved category in the city.

8. Limitations & Future Work

8.1 Current Limitations

- Class imbalance: Arrests are only 13.85% of incidents. Even with balanced class weights, arrest recall is 0.55 - 45% of actual arrests are missed. SMOTE or cost-sensitive learning could improve minority-class recall.
- Population denominator: The 2020 Census residential population is an imperfect proxy for exposure risk. The Loop has far more daytime activity than its 45,396 residents suggest; ambient population estimates (from mobile device data) would give more accurate crime rates.
- Temporal scope: This analysis covers only 2024. A multi-year dataset (2018-2024) would enable trend analysis and detection of pre/post-COVID pattern shifts.
- K-Means assumptions: K-Means forces every point into a cluster and assumes spherical shapes. Chicago's elongated north-south geography may not align well with circular Euclidean clusters. DBSCAN would better handle irregular shapes and identify true noise points.
- Apriori item design: limiting to top-10 crime types drops rarer crime categories from the transaction database. Lower support thresholds with more crime types could reveal additional patterns at the cost of more computation.

8.2 F

- DBSCAN clustering: unlike K-Means, DBSCAN discovers clusters of arbitrary shape and automatically flags isolated incidents as noise - better suited to irregular crime geography.
- Weather data integration: merging daily temperature and precipitation (from NOAA) would test the well-documented hypothesis that crime rates rise in warm weather.
- Multi-year analysis: extending the dataset to 2018-2024 would enable trend modeling and detection of COVID-related crime pattern shifts.
- Real-time dashboard: the trained Random Forest model could be integrated into the existing HTML dashboard, providing police dispatchers with a live probability-of-arrest score.
- FP-Growth algorithm: a more memory-efficient alternative to Apriori that scales better to larger itemsets and lower support thresholds.

9. Conclusion

This project applied a complete data mining workflow to the Chicago Crime 2024 dataset - from raw data ingestion through preprocessing, data integration, and three distinct analytical models. Starting from 259,032 raw records, the pipeline produced a clean, enriched, and analytically ready dataset that supported unsupervised learning, supervised classification, and association rule discovery.

Summary of Deliverables

Deliverable	Outcome
Data Cleaning	257,547 records retained; 99.4% preservation rate; 7 columns dropped
Normalization	Min-Max (Lat/Long for K-Means); Z-Score (District/Ward for PCA)
Dimensionality Reduction	PCA: 4 features -> 2 components, 84.21% variance retained
Discretization	3 techniques: time bins, city sectors (77->4), indoor/outdoor flag
Data Integration	2020 Census demographics merged for all 77 community areas
Model 1 - Clustering	K-Means (K=10) with elbow validation; 10 behaviorally distinct clusters
Model 2 - Classification	Random Forest: 84.29% accuracy, ROC-AUC 0.8154, 12 features
Model 3 - Freq. Patterns	Apriori: 333 itemsets, 316 rules, top lift 2.43 (Motor Vehicle Theft)
Visualizations	11 publication-quality charts embedded in this report
Code	github.com/techsamagan - 5 Python scripts, fully commented

The most important lesson from this project is that crime data requires multiple analytical lenses. Raw counts mislead - per-capita rates reveal equity issues. Geographic clustering shows WHERE to act. Classification shows WHAT predicts whether a crime gets resolved. Frequent Pattern Mining shows WHICH crime-attribute combinations are systematically under-policed. No single model or metric tells the whole story, which is precisely why a structured pipeline that combines all three approaches is necessary for generating actionable, policy-relevant insights from urban crime data.